

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа № 3

Выполнил:

Волков Владимир Алексеевич

Группа:

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Цель лабораторной работы заключалась в миграции ранее разработанного приложения, выполненного в рамках лабораторных работ № 1 и № 2, на фреймворк Vue.JS и в построении одностраничного веб-приложения (SPA). Требовалось подключить роутер, реализовать взаимодействие с внешним API с использованием axios, выполнить разумное деление проекта на компоненты и выделить повторяющуюся логику в composable-файлы. В качестве предметной области был выбран сервис управления личными финансами и бюджетом MoneyFlow.

Ход работы

В ходе выполнения лабораторной работы существующий проект был полностью перенесён на архитектуру Vue 3 с использованием современного подхода к построению SPA-приложений. Вся логика интерфейса была организована в виде маршрутов, представлений, компонентов, composable-файлов, хранилищ состояния и отдельных API-модулей.

1. Организация структуры проекта. В папке src проект был разбит на логические части: api, components, composables, layouts, router, stores, utils и views. Такое разбиение позволило отделить представления страниц от бизнес-логики, кода работы с сервером и повторно используемых частей интерфейса.

2. Подключение роутера. Для маршрутизации был использован Vue Router. В приложении созданы отдельные маршруты для главной страницы, входа, регистрации, личного кабинета, отчётов и интеграций. Также была реализована защита приватных страниц: при отсутствии авторизации пользователь перенаправляется на страницу входа.

3. Работа с внешним API. Для взаимодействия с данными был настроен axios-клиент с базовым адресом локального mock API на JSON Server. Через отдельные модули src/api/users.js и src/api/finance.js реализованы вход и регистрация пользователя, загрузка счетов, бюджетов и транзакций, добавление и удаление операций, импорт демонстрационных транзакций и сохранение правил обработки.

4. Использование хранилищ состояния. Состояние приложения организовано с помощью Pinia. В отдельных store-файлах выделены логика авторизации, финансовых данных, интерфейсного состояния и блока рыночных данных. Это позволило централизовать работу с пользовательскими сущностями и упростило обмен данными между страницами и компонентами.

5. Компонентный подход. Интерфейс был разбит на небольшие переиспользуемые компоненты. Например, BaseNavbar отвечает за навигацию, PasswordField инкапсулирует поле пароля, ToastMessage отображает уведомления, BudgetProgress выводит прогресс бюджета, TransactionTable отвечает за таблицу операций, а LineTrendChart используется для визуализации данных на странице отчётов.

6. Использование composable. Для повторяющейся логики были созданы composable-файлы useTheme, useToast, useAsyncAction и useUserFinanceData. Они отвечают соответственно за переключение темы, показ уведомлений, обработку асинхронных действий и загрузку пользовательских финансовых данных. Это позволило избежать дублирования кода и сделать представления страниц компактнее.

7. Реализованный функционал. В приложении сохранены и доработаны основные функциональные блоки исходного проекта: главная страница с описанием сервиса, вход и регистрация пользователя, личный кабинет со счетами, транзакциями и бюджетами, страница отчётов с графиком, прогнозом и детализированным разбором периодов, а также страница интеграций с демонстрационным импортом транзакций и правилами их обработки. Дополнительно были реализованы валидации форм, защита от повторного импорта одинаковых операций, корректная работа с числовыми паролями и поддержка светлой и тёмной тем.

8. Итоговая проверка требований. По результатам работы было подтверждено, что все условия лабораторной работы выполнены: приложение перенесено на Vue.JS, роутер подключён, работа с внешним API реализована через axios, структура проекта построена на компонентном подходе, а повторяющаяся логика вынесена в composable-файлы.

Вывод

В результате выполнения лабораторной работы было разработано одностраничное веб-приложение MoneyFlow на базе Vue.JS. Миграция исходного проекта на SPA-подход выполнена успешно: реализована маршрутизация, настроено взаимодействие с внешним API, код разделён на компоненты и composable-файлы, а структура проекта стала более модульной и удобной для сопровождения. Следовательно, поставленная задача решена в полном объёме.









